Лабораторная работа №9: Архитектурный рефакторинг проекта: Clean Architecture и подготовка к защите

Курс: Объектно-ориентированное программирование 1 (CS2203)
Университет: КазНУ им. аль-Фараби
Преподаватель: Санжар Аубакиров, PhD
Неделя: 9 из 10
Формат: Лабораторная работа, 3 академических часа (135 минут)
Среда выполнения: Python 3.12+, pytest, Git, VS Code / PyCharm

---

1. Цель и контекст работы

Цель работы: Собрать все разработанные за семестр модули городской транспортной системы Avtobys в целостную систему с чистой многослойной архитектурой (**Clean Architecture**) и подготовить проект к финальной публичной защите на 10-й неделе. Студентам предстоит: 1. Выстроить строгие границы слоев: **Domain**, **Application (Use Cases)**, **Infrastructure** и **Presentation (UI / CLI)** в соответствии с *The Dependency Rule*. 2. Написать архитектурные тесты, гарантирующие независимость доменной логики от баз данных и внешних фреймворков. 3. Устранить оставшиеся запахи кода (**Code Smells**) с помощью объектно-ориентированного рефакторинга. 4. Прогнать сквозной интеграционный сценарий смены автобуса (посадка, оплата по картам/QR, офлайн-очередь, выгрузка отчета).

---

2. Финальная структура репозитория

К концу 9-й недели репозиторий студента должен иметь эталонную структуру:

```
transit_clean_system/
├── domain/                 # Ядро бизнеса (0 внешних зависимостей!)
│   ├── entities/           # Bus, Route, TransitCard, SingleTripTicket
│   ├── value_objects/      # Money, PlateNumber, GeoPoint
│   └── exceptions.py       # Доменные исключения
├── application/            # Сценарии использования (Use Cases)
│   ├── ports/              # Интерфейсы репозиториев, шлюзов, нотификаторов
│   └── use_cases/          # ValidateRideUseCase, IssueTicketUseCase, ShiftReportUseCase
├── infrastructure/         # Внешний мир (реализация портов)
│   ├── persistence/        # SqliteCardRepository, MemoryLedgerRepository
│   ├── adapters/           # BankSdkAdapter, QrPaymentAdapter
│   └── hardware/           # ConsoleAudioVisualDriver, VirtualCardReader
├── presentation/           # Интерфейс пользователя и запуск
│   ├── cli/                # Интерактивная консоль валидатора
│   └── simulator.py        # Скрипт симуляции рейса
├── tests/
│   ├── unit/               # Модульные тесты домена и use cases
│   ├── architectural/      # Тесты чистоты слоев (Dependency Rule)
│   └── integration/        # Сквозные тесты симуляции смены
└── README.md               # Архитектурный паспорт проекта и UML-диаграммы
```

Критерии оценки (20 баллов):

Блок	Задание	Баллы
Блок 1	Рефакторинг слоев Clean Architecture и соблюдение Dependency Rule	6
Блок 2	Разработка архитектурных тестов на чистоту импортов доменного слоя	5
Блок 3	Реализация сквозного интеграционного сценария рейса в simulator.py	5
Блок 4	Оформление архитектурной документации и UML-диаграмм в README.md	4

---

3. Практические задания

Блок 1. Рефакторинг и изоляция слоев (6 баллов)

1. Перенесите сущности Bus, Route, TransitCard в domain/entities/.
2. Замените примитивные типы на **Value Objects**:
    - Вместо str для госномера используйте иммутабельный PlateNumber(value: str) с проверкой регулярного выражения.
    - Вместо сырого Decimal используйте объект-значение Money(amount: Decimal, currency: str = "KZT").
3. В слое application/ports/ объявите абстрактные протоколы портов:
    - CardRepositoryPort
    - PaymentGatewayPort
    - AuditLedgerPort
4. В слое application/use_cases/ напишите сценарий ValidateRideUseCase, который принимает DTO-запрос, вызывает доменные методы и сохраняет результат в репозиторий.

Блок 2. Архитектурные тесты (5 баллов)

В файле tests/architectural/test_clean_rules.py напишите автоматический тест на чистоту архитектурных слоев (инспекция модулей через sys.modules или ast):

```
import ast
import os
import pytest

def test_domain_has_no_infrastructure_imports():
    """Доменный слой не имеет права импортировать инфраструктуру или сторонние библиотеки."""
    domain_path = os.path.join(os.path.dirname(__file__), "../../domain")
    for root, _, files in os.walk(domain_path):
        for file in files:
```

```
            if file.endswith(".py"):
                file_path = os.path.join(root, file)
                with open(file_path, "r", encoding="utf-8") as f:
                    tree = ast.parse(f.read(), filename=file_path)
                for node in ast.walk(tree):
                    if isinstance(node, (ast.Import, ast.ImportFrom)):
                        module_name = getattr(node, "module", "") or ""
                        assert not module_name.startswith("infrastructure"), (
                            f"Нарушение Dependency Rule: {file} импортирует {module_name}"
                        )
                        assert not module_name.startswith("application"), (
                            f"Нарушение Dependency Rule: {file} импортирует {module_name}"
                        )
```

Блок 3. Сквозной интеграционный сценарий (5 баллов)

В presentation/simulator.py реализуйте скрипт полной симуляции смены: 1. Инициализирует репозитории и регистрирует маршрут №32 (10 остановок). 2. Выпускает автобус на рейс. 3. Проводит симуляцию 25 пассажиров (15 стандартных карт, 5 студенческих, 3 QR-кода, 2 карты с нулевым балансом). 4. Эмулирует 1 попытку мошенничества (повторное прикладывание через 15 секунд), которая отклоняется антифрод-цепочкой. 5. Завершает смену и формирует финальный фискальный отчет через ShiftReportUseCase.

Блок 4. Подготовка к защите (4 балла)

В корневом README.md составьте: 1. **UML-диаграмму классов** ядра домена (Mermaid). 2. **UML-диаграмму последовательности (Sequence Diagram)** сценария ValidateRideUseCase. 3. Сводную таблицу устраненных запахов кода (Code Smells) со ссылками на коммиты в Git.

---

4. Чек-лист готовности к защите проекта на Неделе 10

- ☐ Все тесты проходят: pytest tests/ -v (100% passed).
- ☐ Архитектурные тесты подтверждают соблюдение Dependency Rule.
- ☐ В коде отсутствуют заглушки, // TODO, захардкоженные пароли и пути.
- ☐ Проект запускается одной командой: python presentation/simulator.py.
- ☐ Подготовлена презентация с архитектурными решениями и демонстрацией работы системы.